

Atividade Prática Assíncrona

Geomerez Raduan de Oliveira Bandeira

Repositório: https://github.com/raduanoliveira/desenvolvimento_com_ia_aula_assincrona

O texto completo das Etapas 1 a 5 está em `ia-dev-lab/docs/relatorio-etapas.md`. Nesta página estão os quatro pontos pedidos no enunciado.

1. Ferramentas e por quê. Eu configurei o Cursor, versão 3.17.8, porque já o uso no dia a dia e tenho licença paga. Ele cobre a IDE com assistente e o agente no próprio editor. Com ele criei a pasta `ia-dev-lab`, o `hello.py` e o laboratório ToDo: API em Laravel e tela em React, TypeScript, Context API e MUI, tudo no Docker. Também liguei dois servidores MCP: filesystem, com npx, e GitHub, com a imagem oficial no Docker.

2. Trecho útil do AGENTS.md. O trecho que mais me ajudou foi este, nas convenções: *Regra de negócio na camada de aplicação (Service), não no controller e não na tela*. Ele é útil porque a IA tende a colocar a regra no controller quando o pedido é curto. Esse trecho, junto com a seção Não fazer, foi o que eu cobrei de novo no prompt eficaz da Etapa 4. O `AGENTS.md` também traz Sobre o projeto, Comandos, Convenções e Não fazer. Eu conferi se a IA usa esse arquivo com três prompts: subir o ambiente, rodar as checagens automáticas e parar tudo. Os três deram certo. O arquivo completo está em `ia-dev-lab/AGENTS.md`. As regras por pasta estão em `ia-dev-lab/.cursor/rules/`.

3. Prompt fraco versus prompt eficaz. A funcionalidade foi arquivar uma tarefa, sem apagar o registro. O prompt fraco foi só “Adicione arquivar tarefa.” No Grok 4.6 a regra caiu no controller, sem Service e sem teste, e a tarefa arquivada continuou na lista ativa. No Composer 2.5 Fast o código copiou o jeito dos outros Services, mas também pulou os testes. O prompt eficaz disse o contexto, o padrão (teste primeiro, um Service por caso de uso, controller só HTTP), as restrições e como validar. Os dois modelos então criaram um Service novo e escreveram testes. No Grok eficaz passei nove testes da API e sete da tela. A diferença maior foi essa: sem dizer onde a regra vive e como conferir, a IA entrega o caminho mais curto. O comparativo completo, com os dois prompts e os quatro casos, está em `ia-dev-lab/docs/prompts-comparacao.md`.

4. Um obstáculo e como resolvi. O servidor MCP filesystem não subiu, porque esta máquina não tinha npx. Eu instalei o npx no Windows, autorizei os servidores no Cursor e fiz o login do GitHub no navegador. Depois pedi para listar os arquivos do último commit. O MCP do GitHub respondeu: `relatorio.tex` e `relatorio.pdf`. A configuração está em `.mcp.json`, na raiz. Na Etapa 5 eu também abri o Pull Request da branch `feature/setup-inicial` para a `main` e aceitei esse PR direto no GitHub.

Onde está o restante.

Etapa 1: `ia-dev-lab/hello.py` e `ia-dev-lab/docs/relatorio-etapas.md`.

Etapa 2: `ia-dev-lab/AGENTS.md` e `ia-dev-lab/.cursor/rules/`.

Etapa 3: `ia-dev-lab/README.md` e `ia-dev-lab/docs/adr/` (0001 Cursor, 0002 mono-repo, 0003 Docker, 0004 Service).

Etapa 4: `ia-dev-lab/docs/prompts-comparacao.md`.

Etapa 5: PR aceito https://github.com/raduanoliveira/desenvolvimento_com_ia_aula_assincrona/pull/1; PR aberto https://github.com/raduanoliveira/desenvolvimento_com_ia_aula_assincrona/pull/2; `.mcp.json`.

Laboratório: `ia-dev-lab/backend/` e `ia-dev-lab/frontend/`.